Journal of Control, Automation and Electrical Systems
https://doi.org/10.1007/s40313-022-00949-4

Predictive Maintenance: An Embedded System Approach

Igor Tedeschi Franco[1] · Rodrigo Marques de Figueiredo[1]

Received: 13 January 2022 / Revised: 3 July 2022 / Accepted: 18 August 2022
© Brazilian Society for Automatics–SBA 2022

Abstract
Predictive maintenance is an approach growing in industries, mainly in small and medium enterprises, by improving competitiveness. This paper presents a system developed in a mobile edge computing (MEC) architecture. The objective is to unite the benefits of security and the low cost of embedded devices with the necessities of predictive maintenance. This work compares the performance between the embedded and desktop devices and presents a study case considering a programmable logic controller application. For that reason, the predictive system was coded in Python, taking advantage of the language packages. And the case purpose is a simulation scenario of real-time data containing safe operational and error samples, which vary pseudo-randomly. The study result shows that the system achieves high precision. That indicates the viability and the low complexity of applying an embedded device as a solution for predictive maintenance.

Keywords Predictive maintenance · Embedded system · Edge computing · Machine learning · Industry 4.0

1 Introduction

The industrial sector changes over the years, going from steam power to electricity application ending on automation. Nowadays, the new tendency is to apply technology to improve the whole business. This concept is called Industry 4.0. In other words, it is an intelligent use of techniques as a source of information to improve production. For example, predictive maintenance allows each production component to inform when it will have a high probability of failure (Bassi, 2017).

The principle of maintenance starts as a necessary evil, executed only when production stops (Cachada et al., 2018). For this reason, it is responsible for a high operational cost in an industry environment, which can be around 15–60% of the manufacture (Motaghare et al., 2018). Three main categories represent the most common approaches to managing equipment conservation: run-to-failure, preventive maintenance, and predictive maintenance (Susto et al., 2015).

1. *Run-To-Failure (R2F):* is a reactive approach, which performs replacement or repair on failed or broken equipment (Mathew et al., 2017). The main characteristic of this category is to wait until the whole system stops. In other words, even if a small error or fault occurs, the correction will only be applied when the product breaks down (Motaghare et al., 2018).
2. *Preventive Maintenance (PvM):* prevents failure by scheduling some revisions (Susto et al., 2013). For this approach, the planned scheduling considers the number of cycles or times of the life cycle the system or machine will work (Motaghare et al., 2018).
3. *Predictive Maintenance (PdM):* is based on continuous monitoring of a system to maximize its life cycle without breakdown (Motaghare et al., 2018). This approach collects and analyzes the data of equipment automatically, using statistical and machine learning methods (Kovalev et al., 2018).

That last one allows the manufacturer to schedule proactive maintenance, maximizing the equipment operation and reducing downtime and costs. Nevertheless, predictive maintenance goes beyond computational techniques. It implies some managerial and organizational changes: from technological and knowledge improvements to a digital-minded environment culture (Bousdekis et al., 2020).

Small and medium enterprises (SMEs) have more benefits in implementing PdM systems. This kind of manufacturer lost on average 40% of their budget on maintenance. For

✉ Rodrigo Marques de Figueiredo
marquesf@unisinos.br

[1] Control and Automation Engineering, UNISINOS, Avenue Unisinos, 950, Sao Leopoldo, Rio Grande do Sul 93022-750, Brazil

Published online: 01 September 2022

this reason, those enterprises are the main subject for many predictive applications. That goes from a system running on remote servers (cloud computing) to local networks (edge computing), using different techniques, such as machine or deep learning. They all have specific goals, like fault and anomaly detection or a prediction of the remaining useful lifetime (RUL) (Rastogi et al., 2020).

This paper presents the usage of machine learning techniques in a mobile edge computing (MEC) architecture, considering the different approaches to maintenance. This architecture consists of portable devices that support advanced services. That ensures latency, responsiveness, and privacy requirements when compared with cloud services (De Nitto Personè & Grassi, 2019). These devices support complex software in small hardware like smartphones and embedded systems. An embedded operational system (OS) such as Linux provides low cost and reliability once it is an open-source OS. Some very known and powerful devices are based in an embedded Linux: Raspberry Pi®, BeagleBone®, Orange Pi®, etc.

The system focuses on SMEs' necessities. Its main objective is to work as a start point for the organizations that want to enter the Industry 4.0 concept but does not have enough resources. In this way, this paper introduces a simple study case considering an application with a programmable logical controller (PLC). The MEC device is a Raspberry Pi 3 Model B®, based on an embedded Linux distribution called Raspberry Pi OS (previously called Raspbian). The embedded device is responsible for the whole data preparation and prediction. That ensures a minimal impact on the equipment operation, working as a separate component. In this way, it is possible to have a solution with low cost when comparing it with a Cloud platform or a dedicated desktop. The developed PdM system is a prototype to help understand the necessities for each application. It will require some improvements to reach a generic approach.

The paper has six sections, and the first one is this introduction. Section 2 presents the state of the art regarding predictive maintenance. Section 3 introduces the details of the implemented PdM system, considering the preprocessing steps and the algorithms applied. Section 4 details the study case development and how the components communicate with each other. Section 5 shows how the results obtained during the system execution in the study case. Section 6 concludes this paper.

2 Related Works

The industry environment has many difficulties caused by unplanned downtimes. Besides the efficiency and quality of the product manufactured, the cost represents about 15–60% of the whole process (Cheng et al., 2019). The usage of Industry 4.0 technologies has already reached the interest of many manufacturers. This section presents the state of the art of predictive maintenance as the solution for different industrial applications.

The work presented by Liu et al. (2020) develops a platform to predict the mechanical load increase and the shaft misalignment of stepper motors. They implemented an improved support vector machine (SVM) model called least squares support vector machine (LSSVM). The PdM system reached more than 90% of precision.

Several other works take different industrial cases to deploy a solution by predicting some issue. The usage of decision trees to predict failures of a welding machine in a distributed architecture with a programmable logic controller (PLC) and a supervisory control and data acquisition (SCADA) with precision over 85% (Him et al., 2019). Following the distribution system, Cheng et al. (2019) combines two methods to predict both linear (autoregressive integrated moving average—ARIMA) and nonlinear (long short-term memory—LSTM—recurrent neural network) features in a ball bearing production line. They applied the system based on a PLC communicating through MQTT with Cloud, where the prediction occurs.

Besides that, Alves et al. (2020) developed a system of data acquisition and analysis and an intelligent decision support (IDS). An industrial stamping machine controlled by a PLC transmitted the data to the system through MQTT. The system performs the prediction of the next five minutes using the LSTM recurrent neural network. The authors make use of augmented reality as staff support.

Another application is the implementation of ensemble learning techniques to predict the failure of a Wafer Stick Machine in the biscuit industry. The work compares a C4.5 Decision Tree (47.76% of precision) with a predictive system based on four techniques: Lazy locally weighted learning (LWL), Zero-R, J48, and random forest in a hypothetical dataset generated with Monte Carlo. The final result is obtained with a voter to detect the answer, which achieved a precision of 95.90% (Mujib & Djatna, 2020).

Nevertheless, the usage of PdM systems goes beyond an industrial floor. A predictive model applying logistic regression to estimate the failure of an engine in an agricultural environment had an accuracy of 82% (Hivarekar et al., 2020). In the maritime industry, the prediction of corrosion failure in the main engine of vessels was a true challenge for Makridis et al. (2020). They presented a comparison between LSTM, one-class SVM, gradient boosting, and weight permutation entropy to solve this problem, which obtained a precision of 80% only with all algorithms ensembled. A complex system developed using K-means clustering to treat data and an auto-associative neural network, together with a novel key performance indicator (KPI), in a SCADA was applied to identify failures in a wind turbine farm (Gigoni et al., 2019).

It achieved more than 90% of precision according to the operators.

Regarding the embedded system, the main works aimed to support maintenance considers simple and very limited microcontrollers. Hu et al. (2020) and Huang et al. (2020) developed custom hardware platforms based on STM32F4 controllers. Both works considered sensors acquisition with filtering, going through a fast Fourier transform (FFT), and envelope spectrum analysis in mine equipment, the first applied to a motor and the second to a speed reducer. Concari and Bettini (2020) focuses on the limitation of embedded devices, such as memory size and performance of RAM and CPU. They compare three different microcontrollers (STM32F0, STM32F4, and MPC5643) using a rainflow-counting algorithm with some improvements.

In addition to the obstacles faced by predictive maintenance systems, the dataset quality is a challenge. The study published by Silvestrin et al. (2019) compares two deep learning techniques with traditional machine learning models. The main objective of the work was to verify what algorithm performs better with a scarce dataset. They considered the LSTM recurrent neural network and temporal convolutional network (TCN) to represent deep learning techniques. The machine learning models consisted of K-nearest neighbors (KNN), decision tree, and random forest. The decision tree and random forest achieved better performance with few samples, according to the results.

The KNN, decision tree, and random forest were part of another comparative study with SVM, AdaBoost, Naïve Bayers, and XGBoost classifier algorithms. This study compares the performance of two approaches for predictive maintenance: fault-based (FB) and maintenance-based (MB). The work concludes that the MB approach benefits the classification task by having positive results with the decision tree and random forest (Bodo et al., 2020).

The related works presented show some similarities in predictive maintenance applications:

- The increasing usage of deep learning techniques in especial LSTM.
- Traditional machine learning techniques, such as SVM, KNN, and random forest, can solve simple necessities, even with scarce data.
- Distributed systems considering edge computing, SCADA, and other cloud platforms reach the concept of Industry 4.0.
- Systems based on embedded controllers focus on data acquisition or fault analysis and forecast in hostile environments.

This study considers the following algorithms: KNN, SVM, random forest, logistic regression, and dense neural network. The choice of those algorithms takes into account the embedded system limitations and their performance. The device applied in this work was a microprocessor with a Linux operational system (OS). This purpose brings the different methods presented by the authors exposed in this section to a MEC approach.

The project objective is to evaluate the viability of embedded devices in predictive maintenance applications. That is a new approach never studied yet, focusing on a low-cost solution for industries that crave to enter Industry 4.0. The state of the art of prediction in the past years almost always considers clouds or desktop computers, with datasets already obtained. This kind of setup might cost a price high for small and medium enterprises. The proposal brings the known applications for predictive maintenance to a MEC design, taking advantage of the improvements of the embedded devices. The following section details the system developed in the case study presented in this paper.

3 Methodology

Based on the maintenance necessities of the small and medium enterprises, the predictive system developed reaches one of the often applications of this technology: faulty and anomaly detection with machine learning techniques, according to Rastogi et al. (2020). Regarding the architecture, the authors identified that most application focuses on cloud computing. However, for SMEs, this can be a high cost in platforms that have unnecessary processor power. This work considers a mobile edge computing architecture, which is safer than Internet-based approaches. The data can be acquired and processed without leaving the manufacturer's floor, only by applying an embedded system in a local network.

For this reason, the system development has the following requirements:

- Low cost: due to the different economic situations of the enterprises, the system considers open-source technologies, such as Python and Linux. In this way, the cost is only on the device.
- Data management: the PdM must acquire the real-time data automatically without the need for an operator to exchange the data from different machines.
- Auto-adaptation: to learn new features of the application, the predictive system must perform its training again according to some variable (time or number of samples).
- Supervisory: the system should allow the user to check the system anywhere through a supervisory deployed in a local network.

Those four requirements ensure that the predictive system fits the SME's necessities. Besides that, the challenges

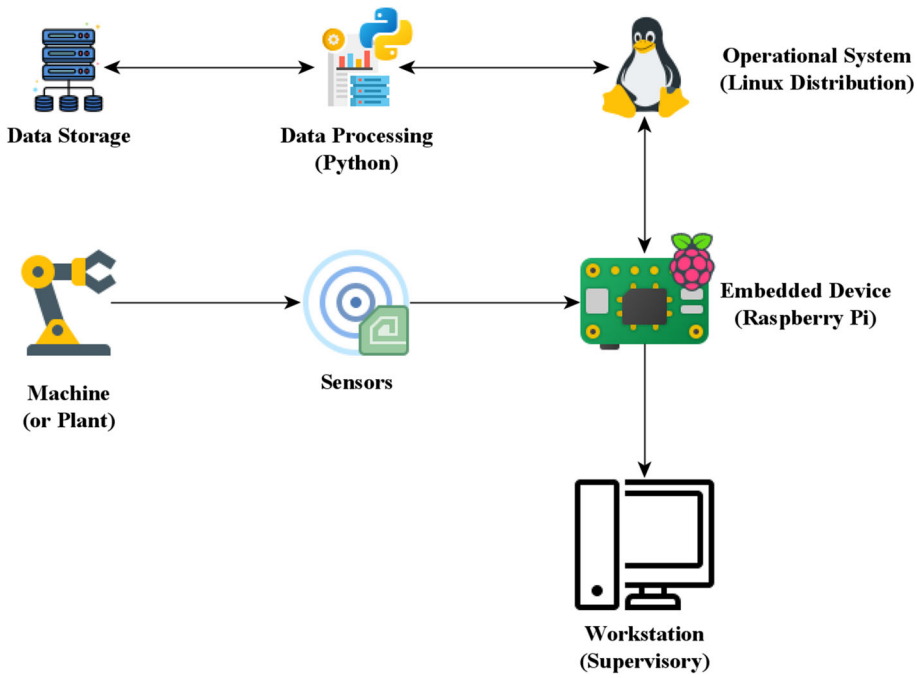

Fig. 1 System overview diagram

of adopting a PdM like data security and management, presented by Bukhsh and Stipanovic (2020), can be overtaken.

To reach the requirements, a Raspberry Pi 3 Model B® received a distribution of Linux with Python 3.8, an interpreted object-oriented programming language. This language has complete packages focused on new software technologies, going from mathematical-graphic resources (similar to MATLAB®) to advanced machine and deep learning techniques. Those packages are kept by a community (open-source) or dedicated companies, constantly improving them. Beyond that, the majority of the resources have their sites, where it is possible to find reference manuals that explain, in detail, how the functions and objects work and examples of them. The Python packages are free, being kept by sponsors and donations.

Figure 1 shows the system diagram, where the computer leaves the function of learning and predicting to just a supervisory, replaced by the embedded device represented by the Raspberry Pi. The data come directly from a machine through sensors or a third-part controller (such as a PLC). In this way, the system based on Linux and Python receives the data in real time and stores it for further historical analysis and performance evaluations.

The PdM system program uses Scikit-Learn (v0.21.3), TensorFlow (v1.11.0), and Pandas (v0.25.1) packages. The Scikit-Learn allows a whole solution of machine learning, from data mining to training and prediction, the same used by Him et al. (2019). The algorithms are optimized to ensure good performance, and the package offers a rich catalog of functions to several techniques: classifiers, linear regressors, clusters, etc. Following the same line as Scikit-Learn, TensorFlow also presents machine learning resources. Even though Scikit-Learn offers a neural network solution, TensorFlow has more elaborated functions. The package delivers variations of neural networks (e.g., recurrent, convolutional, dense). Due to that, only the dense neural network uses TensorFlow. The other algorithms and data mining techniques come from Scikit-Learn.

On the other hand, Pandas is composed of tools that help create databases and treat the samples. It works similar to the known Microsoft® Excel®, commonly used to treat data. The Pandas package also organizes the information in cells, working as an input for the previous packages. Similar to the Microsoft® tool, all the datasets can be imported or exported to a CSV file. Therefore, the embedded device must support the language and resources complexity.

According to Concari and Bettini (2020), the usage of an embedded system brings some limitations, such as random access memory (RAM) size and the processor power. The Raspberry device adopted in this work offers 1GB of RAM and uses a processor with four cores at 1.2GHz. All the operational system (OS) is on a micro SD card with 32GB of memory.

The five algorithms selected at the end of Sect. 2 (SVM, KNN, random forest, logistic regression, and dense neural network) had their performance compared between the embedded device and a desktop computer: Windows(R) 8.1 with 16GB of RAM and a processor with eight cores at 3.6GHz. The results of this test supported the selection of the algorithm that will be part of the PdM system.

The test uses the "Air pressure system failures in Scania trucks" dataset from Kaggle, with 170 features and one label

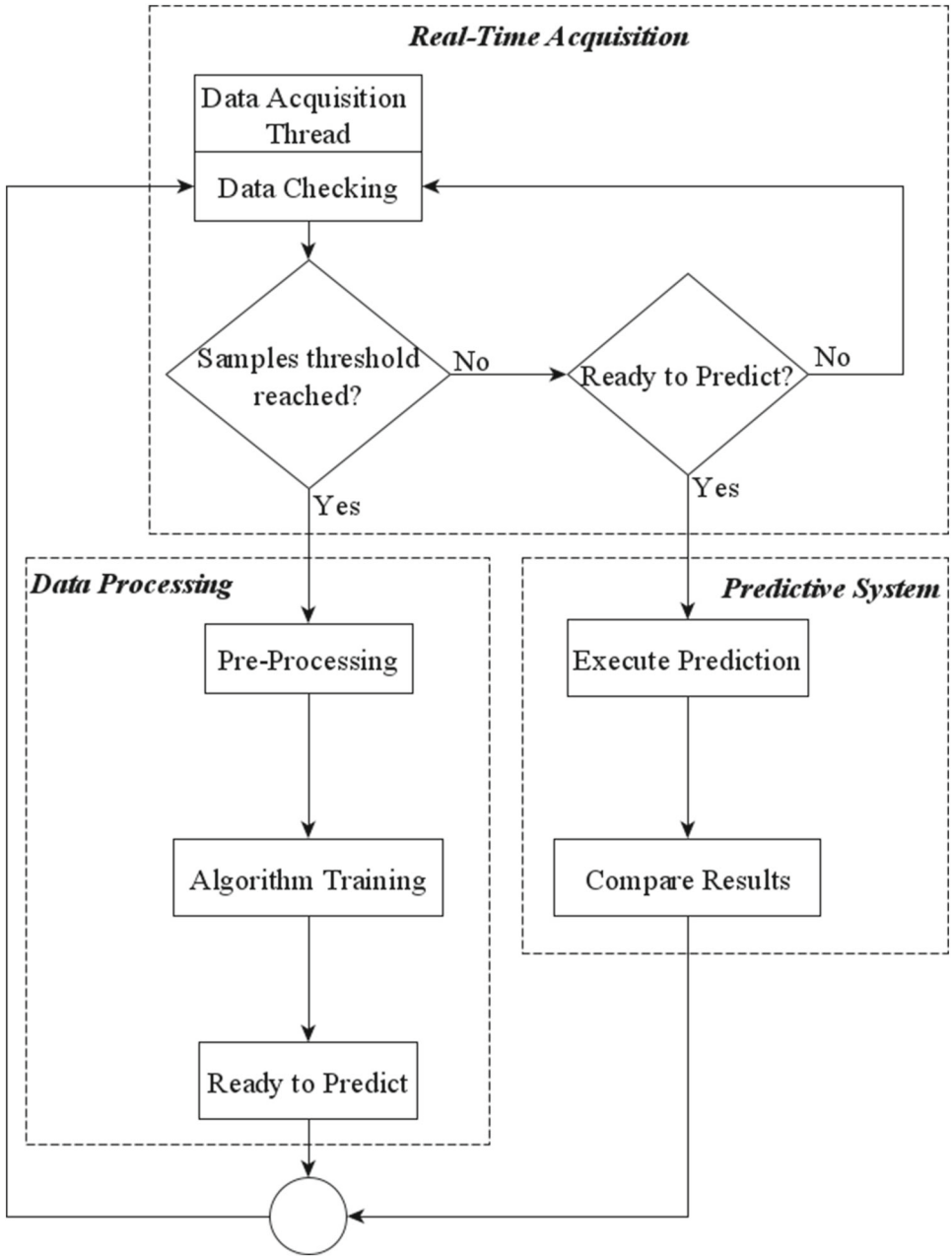

Fig. 2 Predictive system functional diagram

in 76,000 samples. The objective here was to predict failures in the air pressure system on gear changes and braking. The program consists of three steps, where Fig. 2 shows the relationship between them.

1. *Real-Time Acquisition*: a thread dedicated to acquiring the data starts at the beginning of the program. In the test, this thread simulates the real-time samples passing the data from the original file. A mutex controls the data flow during the acquisition task, avoiding the collision of information. A Pandas data frame receives the samples and stores them in a local CSV file. The next step is triggered when the data frame reaches the auto-adaptation threshold (10,000 data entries).
2. *Data Processing*: then, using Pandas, the program transforms all data into numeric values and replaces missing data with the mean of each variable. Next, the MixMaxScaler function of the Scikit-Learn package normalizes the samples to have values between zero and one. At this point, the dataset is split into features and labels and followed by a correlation treatment. The correlation has the goal of optimizing the features. It uses the Pandas resource of correlation and simple for loops to identify the highest correlation (over 90%) with the lower variance of the pair of variables, deleting it. In the tests, two datasets are gen-

Table 1 Algorithm comparison: desktop: eight-core @ 3.6 GHz with 16GB RAM X embedded: four-core @ 1.2 GHz with 1GB RAM

Algorithm	Device	Time [s]	Precision [%]
ANN	Desktop	553.44110	99.00000
	Embedded	1202.88138	98.98095
SVM	Desktop	0.41141	98.85714
	Embedded	3.88431	98.85714
RF	Desktop	2.27534	99.00000
	Embedded	15.63011	99.00000
KNN	Desktop	2.34916	98.71429
	Embedded	29.59829	98.71429
LR	Desktop	0.06506	98.85714
	Embedded	3.44306	98.85714

erated with a static random seed using the train_test_split function of the Scikit-Learn package: 70% for training and 30% for test datasets. The fixed seed value ensures that both the embedded and the desktop have the same dataset division, which guarantees a fair comparison between the devices. After that, the algorithms perform their training and prediction. Each algorithm has its function to create the model, fit the training dataset, and predict the test dataset. The input parameters of each algorithm were: 5,000 iterations for SVM, a value of 500 for the maximum depth of the tree for random forest, three nearest neighbors for KNN, 5,000 iterations for the logistic regression, and one hidden layer with the same number of features for the ANN. The other parameters kept default values. For comparison, the time that each algorithm takes to perform the whole process was measured. To support the algorithm selection, each algorithm received the same dataset. That allows a fair comparison of time and precision, showing the performance of each algorithm on different platforms. For the application, the data processing stage is responsible only to train the model chosen with the number of samples established as the threshold.

3. *Predictive System*: in this stage, the model is already trained and ready to predict. The output of this process is the classification report and the confusion matrix, obtained through the Scikit-Learn package. Considering the objective of this work, the prediction was compared with the label variable (responsible for identifying a failure in the system) for both applications, the algorithm selection tests, and the study case. That supports the performance analysis of the system, helping to verify the capability of positive forecastings. The single difference of this stage in the study case is an MQTT publishment to inform the supervisory of the forecasting failure rate.

The program described above runs on desktop and Raspberry platforms, performing seven training and tests for each algorithm. Table 1 shows the average time and precision of each model on both platforms. It is possible to see that the best three algorithms were SVM, random forest, and logistic regression, which had the lowest time with the highest precision. Figure 3 reveals a confusion matrix of the three algorithms.

A confusion matrix consists (from left to right and top to bottom) of true positives, false positives, false negatives, and true negatives. The precision exposed in Table 1 is the sum of the true values; in other words, the number of times the system predicts a correct positive or negative value. The false values are the incorrect forecasts (e.g., system indicates positive, but the real value is negative). Analyzing Fig. 3, the logistic regression (a) performs well with positive values, failing more than 1% in detecting negatives, which is the same observed for the SVM (c). Only the random forest (b) shows failures lower than 1%. Therefore, considering the balance of positive and negative forecasting, the study case system is based on the random forest model.

According to Liu et al. (2012), the random forest is an ensemble classifier consisting of decision trees. Each tree has the right to vote for the best classification. In the end, the result comes from the highest voted value. This algorithm limits the generalization error, which avoids over-fitting. Liu

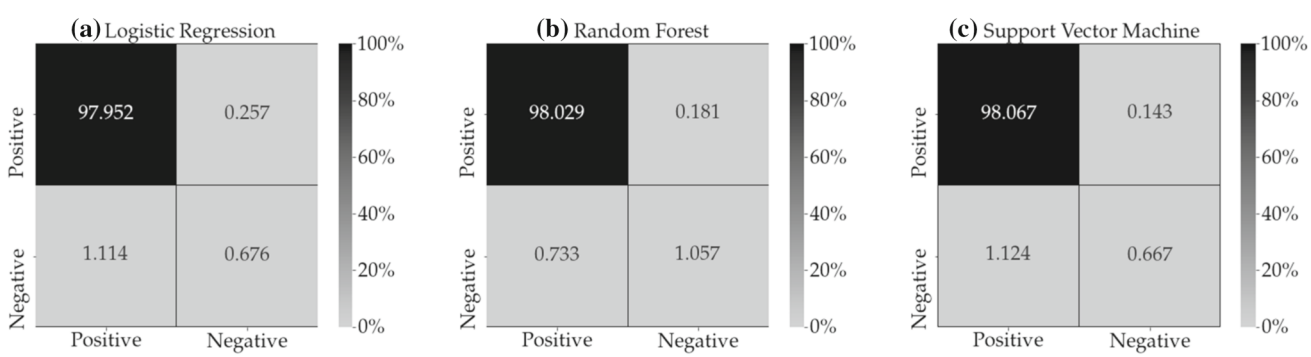

Fig. 3 Confusion matrix comparison

et al. (2012) have already observed the same comparison presented above: the RF has better performance than SVM and Logistic models. Therefore, the great qualities of the chosen model ensure a fast, high performance between the other algorithms. Section 4 will explain some changes in the code to work with a supervisory and a real-time application.

4 Study Case

The random forest has achieved the best precision versus time benefits on both platforms. For that reason, the predictive maintenance system applied considers only that model in the study case. The study case presents a simulated application with a programmable logic controller. Usually, manufacturers make use of PLCs to control their plants or to be part of a machine. This kind of intelligent device allows the user to perform synchronized actions, emergency procedures, etc. As a result, a considerable amount of data passes through the controllers. Hence, that permits a PdM system implementation as an extension of the PLC without significant changes. The studies presented by Him et al. (2019), Alves et al. (2020), and Cheng et al. (2019) worked with PLCs as the first step of data acquisition.

This paper adopts a PLC with integrated analog and digital inputs and outputs, the XP325 of the Nexto Xpress series of Altus®. This PLC has five analog inputs and four outputs, which are configurable to measure current or voltage. The controller's objective is to have a loopback (inputs and outputs directly connected) to simulate engine data. In this way, it can read the analog inputs data written by the application. The XP325 also has support for the MQTT protocol (Cheng et al., 2019; Alves et al., 2020), which allows a safe connection through transport layer security (TLS).

The data acquisition step of the PdM system requires a modification to support MQTT communication, which is publish-subscribe. Each MQTT client connects to a broker (server) responsible for receiving the published messages and forwarding them to the subscribed devices. As mentioned, Python has a lot of packages that help the development. The PAHO-MQTT package (v1.5.0) has the resources needed for a client. It offers functions to publish and subscribe, besides callbacks to receive the messages at the time. Therefore, the internal thread of the packages replaces the logic to acquire the data, where both PLC and Raspberry communicate through a local mosquitto broker (mosquitto.org) installed on a desktop. The mosquitto is an open-source message broker and client. This protocol consists of a publish–subscribe communication, where the Raspberry subscribes to a topic that the controller will publish.

The PLC application design is shown in Figure 4. The variables related are four analog inputs, two configured as voltage (0 to 10V) and the other two as current (0 to 20mA),

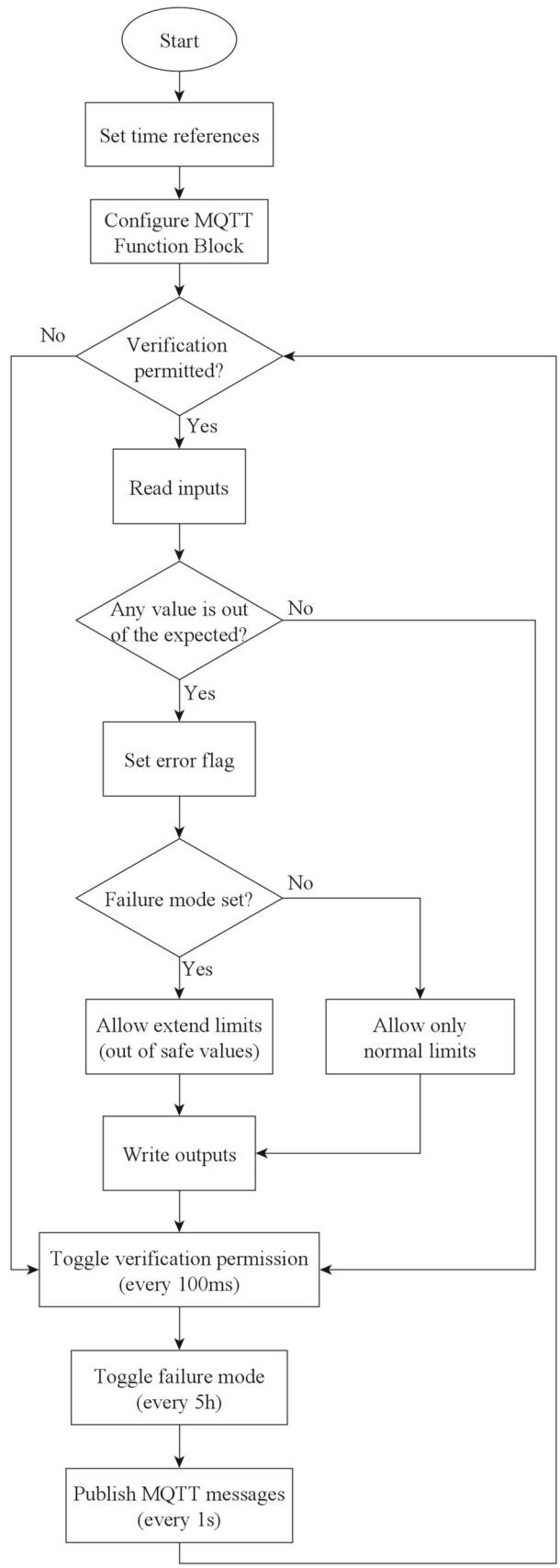

Fig. 4 XP325 application workflow

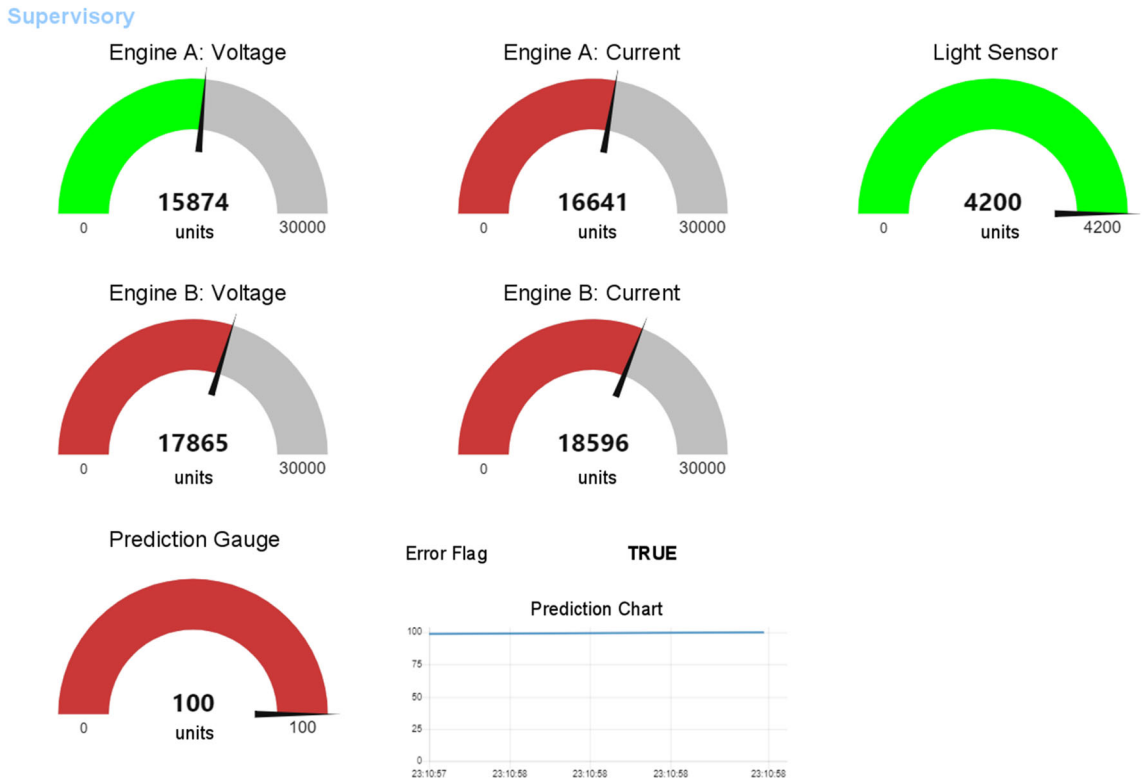

Fig. 5 Supervisory layout design

and a light-dependent resistor (LDR) in a resistance temperature detector (RTD) input. The LDR represents a constant value in the dataset once the system operates in a dark environment. The analog inputs receive the value directly from the outputs.

It reads the analog inputs in values from 0 to 30,000 units, representing the scale (0 to 10 V for the voltage and 0–20 mA for the current) in engineering units (e.u.). The analog outputs increase or decrease pseudo-randomly. This variation consists of a mechanism based on the current time in nanoseconds. The pseudo-mechanism ensures that the user does not know to what value the input will change. The output increases in 100 engineering units (around 33 mV for voltage and 66 uA for current) if the time value is even. On the other hand, an odd value decreases the output in the same proportion. Meanwhile, the RTD inputs must indicate a constant value of 4200 ohms in the shadow.

The controller performs anomaly detection through an error variable. The error goes to TRUE when any input value passes a minimum or maximum specification. These parameters, often provided by the equipment manufacturer as electrical specifications, represent the boundaries of a safe operation. For the study case, the safe values for the voltage were between 8400 and 16,500 e.u. (2.8–5.5 V); for current, between 7500 and 15,000 e.u. (5–10 mA); and for the RTD, above 1500 ohms. As shown in Fig. 4, the application has two operational modes: a normal one and a failure mode. In the normal, the analog output values always respected the safe operational boundaries. However, in the failure mode, the output limits extend. The voltage can be between 6000 and 18,000 e.u. (2–6 V); the current can go from 3000 to 22,500 e.u. (2–15 mA).

All the input variables represent the features of the system, while the error is the label. An MQTT function block in the application publishes the sample to the embedded device, where the PAHO-MQTT functions receive and store each entry in a file. In contrast to the test performed with the Kaggle® dataset, the threshold to start training the random forest model was 50,000 samples. The algorithm updates at approximately 13.8 h, considering that the PLC publishes new data entries at each 1 second, while the modes toggle every 5 hours. Therefore, the dataset used to train the random forest will have behaviors with and without failures.

Besides that, as a requirement for the PdM system, a supervisory based on Node-RED® was built. The Node-RED® allows for creating dashboards that help monitor the current state of a system (Alves et al., 2020). The dashboard is in the same desktop responsible for providing the mosquitto broker, getting the data through MQTT by subscribing to the same topics used between PLC and Raspberry. Thus, it is only available in the local network. For the study case, the supervisory layout is shown in Fig. 5, where all the features are

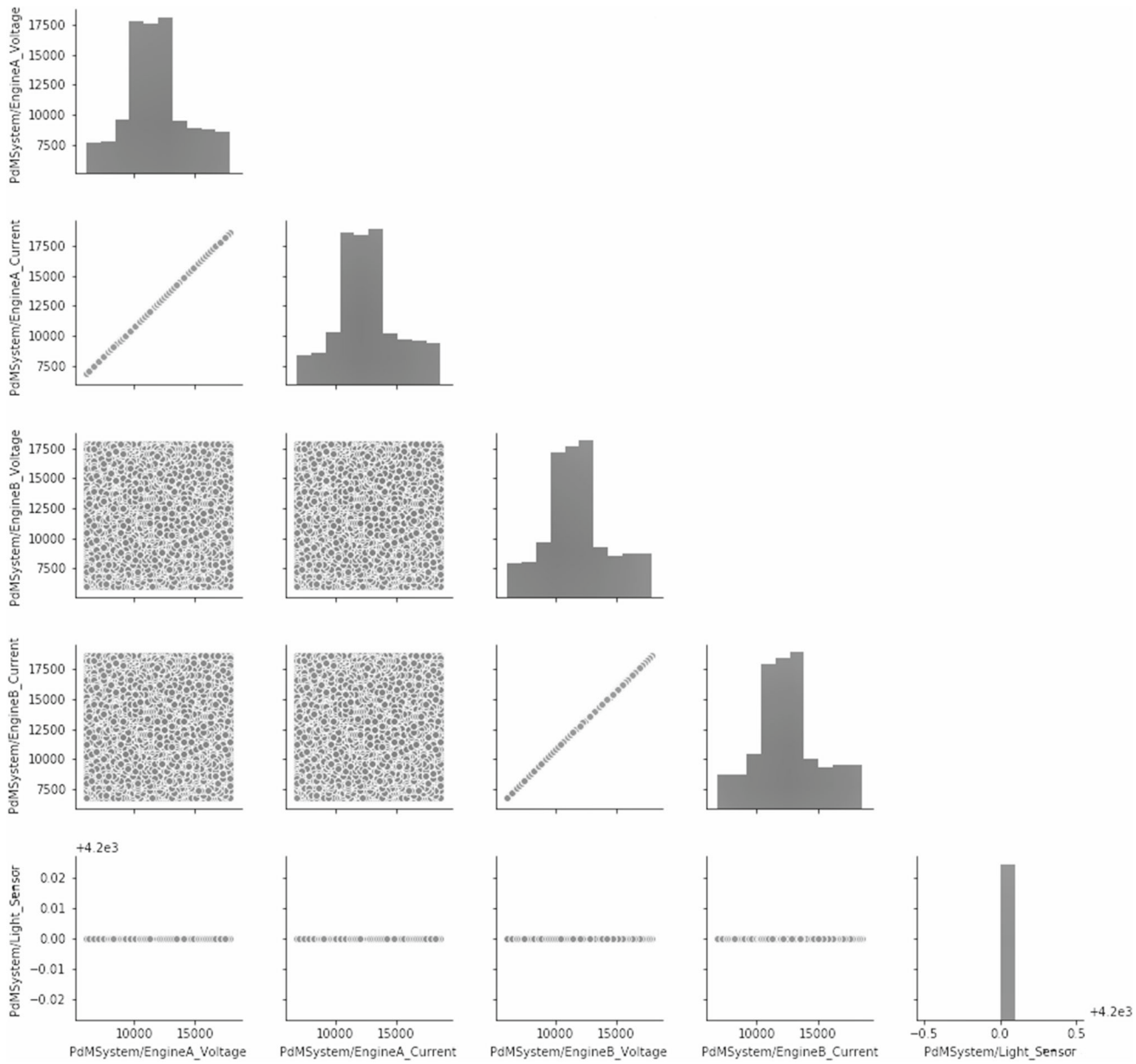

Fig. 6 Data feature charts: the main diagonal shows the distribution of each system input variable, while the graphs below represent the correlation between pairs of features

gauges showing the current values of the analog and RTD inputs. An error flag with TRUE or FALSE represents the label. A prediction gauge increases or decreases every time the random forest predicts an error or not, while the chart shows its history.

All the forecasts and label values were stored in files by the PdM system. Due to availability and time issues, the study case restricts to a viability test for predictive maintenance. Section 5 discusses the datasets and the prediction files stored along four days of execution. Additionally, it brings the results of viability, according to the performance of the system and the requirements presented in Sect. 3.

5 Results

The system ran for four days, acquiring a total of 340,870 samples for each variable: voltage, current, and LDR values. All the data were organized into seven datasets, applied to train the random forest. Considering the train occurred at every 50,000 samples, the program only started to perform predictions after approximately 13, 8 h of execution. In other words, after learning from the first samples, the sys-

tem predicts each new entry until the threshold value, where another training is performed. That indicates seven training with the data acquired while the system was predicting. Once the system mode toggles every 5 h, allowing failures, each dataset has a mix of negative and positive values. The fact that the datasets have distinct behaviors differs from the test of Sect. 3, where the error samples were scarce.

Therefore, the output was six files containing the current error flag value for that entry and the forecasting for that same feature variables. The system made 286,834 predictions over the four days. This number is 15.85% lower than the data obtained, showing a loss of data considered in the prediction stage. That occurred due to the acquisition not stopping during the training, causing some data to not be evaluated.

Figure 6 shows the data distribution and correlation graphs. It is possible to verify that all the features presented a Gaussian distribution, except the light sensor (LDR) that keeps a constant as expected. Also, looking at the graphs below the main diagonal, we expose the correlation between the features, which have three different behaviors:

1. Current x voltage of the same engine: there is a total correlation between them, represented by a straight line growing along the time (correlational diagonal).
2. Current x voltage of different engine: that shows dispersion graphs, indicating that both variables are independent.
3. Light sensor x any other variable: it is a constant giving no impact on the other variables (no-correlated).

These three different behaviors stimulate the data treatment developed for the PdM system to remove a variable of each voltage–current pair due to high correlation. That step enhanced the job performed by the random forest, optimizing its capability to forecast the value based on the current features. Another thing that helps any machine learning algorithm to learn better is the presence of failures. The simulated scenario excited errors in each dataset used in training. That is not the usual situation in a manufacturer situation. Failures do not occur every time. However, considering that the objective is to check the viability of the PdM system approach adopted, that is not a concern of this work. An ideal model helps to understand if an embedded device can replace a computer in this task. Besides that, the data changes pseudo-randomly, and the random forest training always re-executes after the samples threshold is reached. That guarantees the device does not know the data. As a consequence of the ideal scenario presented in the study case, the random forest could achieve a precision of 100%. Figure 7 shows the confusion matrix of the model.

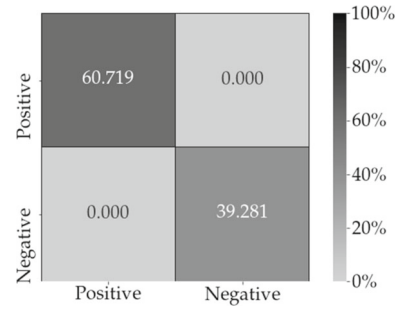

Fig. 7 Embedded predictive system confusion matrix result

The values are distributed in 60% of positives and 40% of negatives. This behavior diverges from Sect. 3, which was approximately 98 and 2%, respectively. According to the characteristics of the random forest, it is known that the system does not suffer an over-fitting. The explanation behind the algorithm's success is the amount of negative data and the Gaussian distribution in Fig. 6. Similarly to that, Fig. 8 shows some examples of distributions found in the Scania dataset used in Sect. 3.

The behavior in Fig. 8 is far from a Gaussian model. That justifies the different accuracies of the random forest between the test and the study case. Also, if the SVM and logistic regression were applied, they probably would have a good performance but can be over-fitted.

Therefore, the result shows that applying an embedded device to a production line that already uses a PLC is not complicated. The PdM system purposed in the study case achieved all the requirements of Sect. 3. The whole system was developed with open-source software, causing the only cost remaining to the embedded device selected. Also, the usage of Python with the packages dedicated to machine learning, such as Scikit-Learn, makes the implementation of an algorithm much easier. Moreover, the MQTT protocol (through the PAHO-MQTT package) facilitates the work to receive the data in real time with simple functions, allowing the communication with a supervisory via Node-RED(R) in a local, safe network.

This work accomplished its objective of presenting a cheap way to enter Industry 4.0 concepts, considering the different realities of small and medium enterprises. The study case focuses on checking the viability of using an embedded device dedicated (MEC) to the function instead of a central computer. That is a simple way to enter Industry 4.0 without high investments. The concept of prediction is constantly improving, and the tools to reach it are not so complex. The presented system showed some free and easy-to-use resources that allow the industries to start their predictive applications.

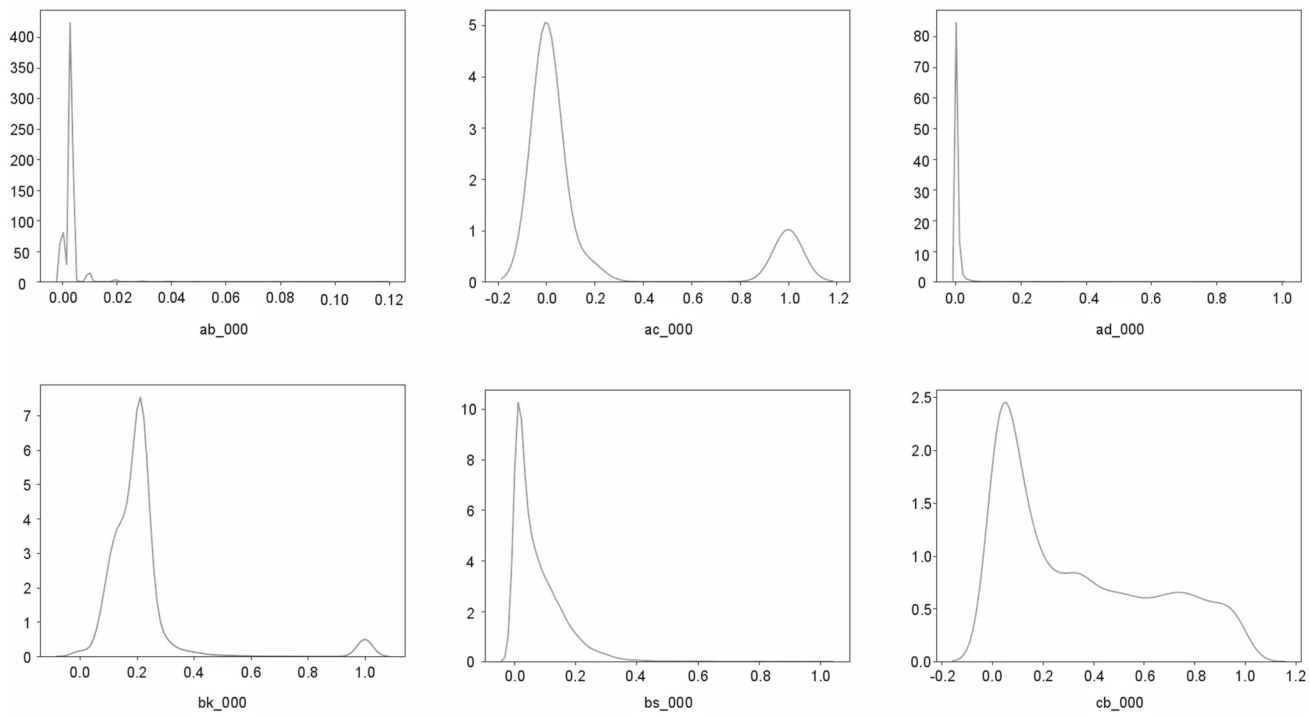

Fig. 8 Scania dataset distribution examples

6 Conclusion

The predictive maintenance increased and evolved over the years. Its applications took several advantages to industries, mainly the small and medium enterprises, which have great competitive necessities (Rastogi et al., 2020). However, applying this Industry 4.0 concept requires some economic and cultural changes in the whole organization (Bousdekis et al., 2020). This work presented an easy solution to reach this goal.

The designed system was developed over open-source software, utilizing Linux as the operational system and programmed with Python. Packages like Scikit-Learn, Pandas, PAHO-MQTT, and others made the task easier. Taking advantage of these resources allowed a predictive program to be built without much effort and knowledge. The study case presented in this work applies those resources in an embedded system to ensure data security and a low-cost solution: a mobile edge computing architecture (De Nitto Personè & Grassi, 2019).

The methodology approach in this work was started by showing the performance differences between a desktop computer and an embedded device. From this comparison, random forest showed the best time and precision relation. The predictive system ran in a simulated scenario based on an application with a PLC communicating through MQTT (Cheng et al., 2019; Alves et al., 2020). The study case considered pseudo-random behavior to change the feature values. The PLC signed an error flag (label variable), which could raise if the value overpassed the safe operational boundaries. With the dataset number of samples (over 340,000) and errors occurrences, the model reached a precision of 100%. The excellent result comes from the high performance of the random forest, which avoids over-fitting (Liu et al., 2012), besides the Gaussian distribution of the dataset, favoring the algorithm.

The embedded system could accomplish the goal of showing the necessary to apply predictive maintenance. The resources presented in this work have good documentation and examples, allowing the developer to understand their functionalities. Therefore, entering this Industry 4.0 concept can be less traumatic than expected. Applying predictive maintenance is still growing, and the resources follow this evolution, allowing small and medium enterprises to evolve together.

The state of the art of predictive maintenance is compound by applications considering computers. Just a few authors like Hu et al. (2020), Huang et al. (2020) and Concari and Bettini (2020) used embedded devices for forecasting, but with microcontrollers instead of microprocessors based on Linux. The present work introduced a new approach to imple-

menting in a manufacturing plant with low impact but with all benefits of the predictive maintenance.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Alves, F., Badikyan, H., Antonio Moreira, H. J., Azevedo, J., Moreira, P. M., Romero, L., & Leitao, P. (2020). Deployment of a smart and predictive maintenance system in an industrial case study. In *IEEE international symposium on industrial electronics* (Vol. 2020). https://doi.org/10.1109/ISIE45063.2020.9152441

Bassi, L. (2017). Industry 4.0: Hope, hype or revolution? In *RTSI 2017—IEEE 3rd international forum on research and technologies for society and industry, conference proceedings*. https://doi.org/10.1109/RTSI.2017.8065927

Bodo, R., Bertocco, M., & Bianchi, A. (2020). Fault classification driven by maintenance management for smart maintenance applications. In *2020 IEEE international workshop on metrology for Industry 4.0 and IoT, MetroInd 4.0 and IoT 2020—Proceedings*. https://doi.org/10.1109/MetroInd4.0IoT48571.2020.9138294

Bousdekis, A., Apostolou, D., & Mentzas, G. (2020). Predictive maintenance in the 4th industrial revolution: Benefits, business opportunities, and managerial implications. *IEEE Engineering Management Review*. https://doi.org/10.1109/EMR.2019.2958037

Bukhsh, Z. A., & Stipanovic, I. (2020). Predictive maintenance for infrastructure asset management. *IT Professional*. https://doi.org/10.1109/MITP.2020.2975736

Cachada, A., Barbosa, J., Leitno, P., Gcraldcs, C. A. S., Deusdado, L., Costa, J., Teixeira, C., Teixeira, J., Moreira, A. H. J., Moreira, P. M., & Romero, L. (2018). Maintenance 4.0: Intelligent and predictive maintenance system architecture. In *2018 IEEE 23rd international conference on emerging technologies and factory automation (ETFA)* (pp. 139–146). IEEE. https://doi.org/10.1109/ETFA.2018.8502489

Cheng, C., Zhang, B. K., & Gao, D. (2019). A predictive maintenance solution for bearing production line based on edge-cloud cooperation. In *Proceedings—2019 Chinese automation congress, CAC 2019*. https://doi.org/10.1109/CAC48633.2019.8996482

Concari, C., & Bettini, G. (2020). Embedded implementation of rainflow-counting for on-line predictive maintenance. In *ECCE 2020—IEEE energy conversion congress and exposition*. https://doi.org/10.1109/ECCE44975.2020.9236298

De Nitto Personè, V., & Grassi, V. (2019). Architectural issues for self-adaptive service migration management in mobile edge computing scenarios. In *Proceedings—2019 IEEE international conference on edge computing, EDGE 2019—Part of the 2019 IEEE world congress on services*. https://doi.org/10.1109/EDGE.2019.00020

Gigoni, L., Betti, A., Tucci, M., & Crisostomi, E. (2019). A scalable predictive maintenance model for detecting wind turbine component failures based on SCADA data. In *IEEE power and energy society general meeting* (Vol. 2019). https://doi.org/10.1109/PESGM40551.2019.8973898

Him, L. C., Poh, Y. Y., & Pheng, L. W. (2019). IoT-based predictive maintenance for smart manufacturing systems. In *2019 Asia-Pacific signal and information processing association annual summit and conference, APSIPA ASC 2019*. https://doi.org/10.1109/APSIPAASC47483.2019.9023106

Hivarekar, N., Jadav, S., Kuppusamy, V., Singh, P., & Gupta, C. (2020). Preventive and predictive maintenance modeling. In *Proceedings—Annual Reliability and Maintainability Symposium* (Vol. 2020). https://doi.org/10.1109/RAMS48030.2020.9153636

Hu, W., Zhou, L., Du, J., Li, S., & Kong, C. (2020). Mine motor predictive maintenance monitoring equipment and algorithm design based on STM32. https://doi.org/10.1109/ITAIC49862.2020.9338853

Huang, M. T., Li, S., Xue, Y., & Li, S. Y. (2020). Design of embedded hardware platform for predictive maintenance of coal mine reducer. In *Proceedings—2020 5th international conference on mechanical, control and computer engineering, ICMCCE 2020*. https://doi.org/10.1109/ICMCCE51767.2020.00237

Kovalev, D., Shanin, I., Stupnikov, S., & Zakharov, V. (2018). Data mining methods and techniques for fault detection and predictive maintenance in housing and utility infrastructure. In *2018 international conference on engineering technologies and computer science (EnT)* (pp. 47–52). IEEE. https://doi.org/10.1109/EnT.2018.00016. https://ieeexplore.ieee.org/document/8420112/

Liu, Y., Wang, Y., & Zhang, J. (2012). New machine learning algorithm: Random forest. In *Lecture notes in computer science (including subseries lecture notes in artificial intelligence and lecture notes in bioinformatics)* (Vol. 7473). LNCS.

Liu, Z., Yuan, T., Zhou, X., Yuan, X., Wang, Y., & Zhang, X. (2020). Research on predictive maintenance technology of stepping motor based on load value analysis. In *Proceedings—2020 Chinese automation congress, CAC 2020*. https://doi.org/10.1109/CAC51589.2020.9327408

Makridis, G., Kyriazis, D., & Plitsos, S. (2020). Predictive maintenance leveraging machine learning for time-series forecasting in the maritime industry. In *2020 IEEE 23rd international conference on intelligent transportation systems, ITSC 2020*. https://doi.org/10.1109/ITSC45102.2020.9294450

Mathew, J., Luo, M., & Pang, C. K. (2017). Regression kernel for prognostics with support vector machines. In *2017 22nd IEEE international conference on emerging technologies and factory automation (ETFA)* (pp. 1–5). IEEE. https://doi.org/10.1109/ETFA.2017.8247740

Motaghare, O., Pillai, A. S., & Ramachandran, K. I. (2018). Predictive Maintenance Architecture. In *2018 IEEE international conference on computational intelligence and computing research, ICCIC 2018*. https://doi.org/10.1109/ICCIC.2018.8782406

Mujib, A., & Djatna, T. (2020). Ensemble learning for predictive maintenance on wafer stick machine using IoT sensor data. In *2020 international conference on computer science and its application in agriculture, ICOSICA 2020*. https://doi.org/10.1109/ICOSICA49951.2020.9243180

Rastogi, V., Srivastava, S., Mishra, M., & Thukral, R. (2020). Predictive maintenance for SME in Industry 4.0. In *Proceedings—2020 global smart industry conference, GloSIC 2020*. https://doi.org/10.1109/GloSIC50886.2020.9267844

Silvestrin, L. P., Hoogendoorn, M., & Koole, G. (2019). A comparative study of state-of-the-art machine learning algorithms for predictive maintenance. In *2019 IEEE symposium series on computational intelligence, SSCI 2019*. https://doi.org/10.1109/SSCI44817.2019.9003044

Susto, G. A., Schirru, A., Pampuri, S., Pagano, D., McLoone, S., & Beghi, A. (2013). A predictive maintenance system for integral type faults based on support vector machines: An application to ion implantation. In *2013 IEEE international conference on automation science and engineering (CASE)* (pp. 195–200). IEEE. https://doi.org/10.1109/CoASE.2013.6653952

Susto, G. A., Schirru, A., Pampuri, S., McLoone, S., & Beghi, A. (2015). Machine learning for predictive maintenance: A multiple classifier approach. *IEEE Transactions on Industrial Informatics, 11*(3), 812–820. https://doi.org/10.1109/TII.2014.2349359.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.